CV5570 – Computer Vision for Construction Engineering

Object Detection



Assignment 02

By

Vasantharan K
ID26S012

Date: 05.10.2026

Assignment 2

CV5570 – Computer Vision for Construction Engineering

Objective

The objective of this assignment is to develop and evaluate an object detection pipeline for construction equipment using three different detector families: a two-stage detector, a one-stage detector, and a transformer-based detector.

The **Annotated Construction Images Dataset (ACID)** is used for the experiments. ACID was developed specifically for construction-machine detection and contains images of construction equipment with manually annotated object locations. The original ACID study reported 10,000 images covering 10 construction-machine categories and evaluated several object detection algorithms including YOLOv3, Inception-SSD, R-FCN-ResNet101, and Faster R-CNN-ResNet101.

For this assignment, the experiments are restricted to three classes:

1. Excavator
2. Dozer
3. Dump Truck

The following three detectors are implemented:

- **Two-stage:** Faster R-CNN with ResNet-50-FPN
- **One-stage:** YOLOv8s
- **Transformer-based:** DETR with ResNet-50

The models are compared using detection accuracy, per-class average precision, inference speed, parameter count, training time, convergence behaviour, and qualitative success and failure cases.

The main objective is not only to determine which detector achieves the highest accuracy, but also to understand the practical trade-off between **accuracy, computational cost, inference speed, and failure behaviour** for construction-site monitoring.

Images/Dataset Description, Acquisition and Preprocessing

ACID Dataset

The ACID dataset is a construction-machine image dataset developed for deep-learning-based object detection. The dataset provides bounding-box annotations for construction machinery and was designed to support applications such as construction productivity monitoring and safety

monitoring. The official ACID website states that annotations follow consistency, correctness, and completion standards, including annotation of visible and occluded construction machines.

For this assignment, a three-class subset was extracted containing **3,929 images and 7,614 annotated object instances**.

The annotation data were available in both CSV and JSON formats. The CSV file was used as the primary annotation source during preprocessing because the bounding-box coordinates were directly available in tabular form.

The CSV annotation contains the following information: ***"Image filename, Image width, Image height, Object class, xmin, ymin, xmax and ymax."***

The JSON annotation follows a structured image-category-annotation organization.

Dataset Statistics

Class	**Images Containing Class**	**Instances**
Excavator	2,239	2,790
Dozer	1,025	1,123
Dump Truck	2,206	3,701
Total	**3,929 unique images**	**7,614**

The dataset therefore has a moderate class imbalance. Dump Truck is the most frequent class with 3,701 instances, while Dozer has the fewest instances with 1,123.

The class imbalance is important when interpreting the detector results because a model can achieve good overall performance while performing substantially worse on the less frequent Dozer class.

Bounding Box Analysis

For every annotation, the following quantities were calculated:

- Bounding-box width
- Bounding-box height
- Bounding-box area
- Bounding-box aspect ratio
- Bounding-box area relative to image area

The bounding-box width ranged from **12 to 3,768 pixels**, while the height ranged from **10 to 2,696 pixels**.

The mean bounding-box width was approximately **391.45 pixels**, and the mean height was approximately **271.64 pixels**.

The mean bounding-box aspect ratio was **1.51**, with values ranging from **0.164 to 20.90**.

The mean bounding-box area ratio was approximately **0.168**, indicating considerable variation in the relative size of construction equipment within images.

These variations make the dataset suitable for comparing detectors under different object scales and aspect ratios.

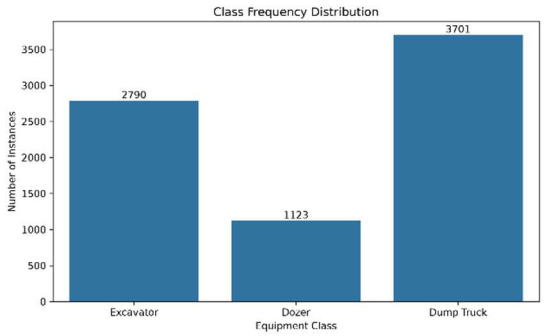


Fig 1. Class frequency distribution

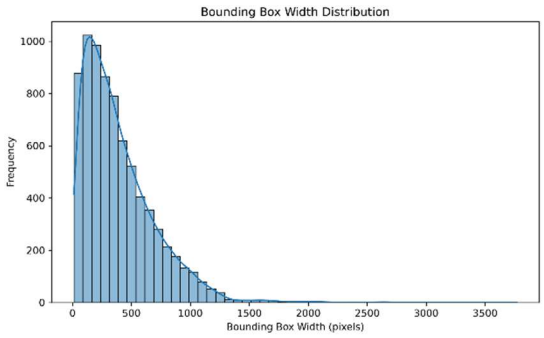


Fig 2. Bounding-box width distribution

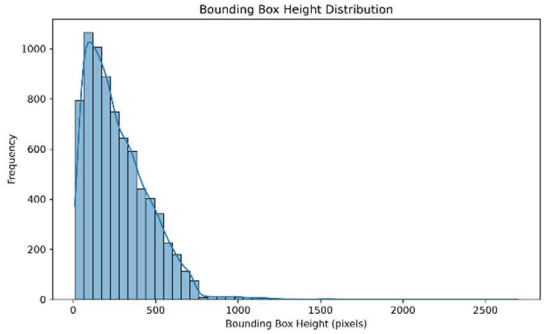


Fig 3. Bounding-box height distribution

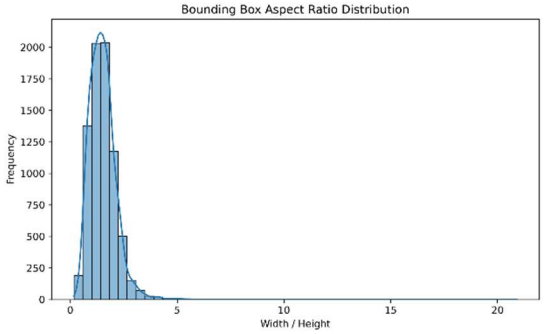


Fig 4. Bounding-box aspect ratio distribution

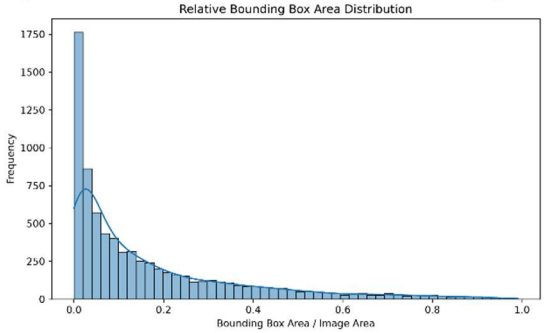


Fig **5.** Relative bounding-box area distribution

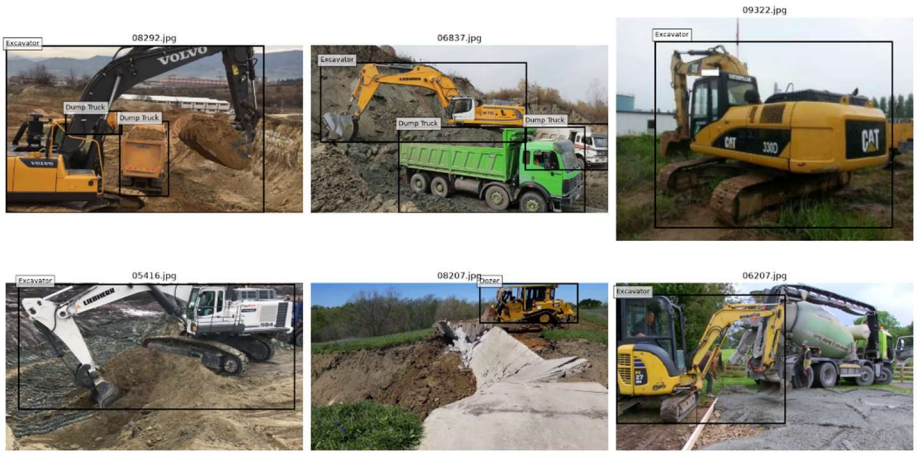


Fig 6. Six sample images with ground-truth bounding boxes

Train/Validation/Test Split

The dataset was divided at the **image level** into training, validation, and test sets using a **70:15:15 ratio**.

A fixed random seed of **42** was used to make the split reproducible. Splitting was performed at the image level rather than at the bounding-box level so that annotations from the same image could not appear in different subsets.

The resulting split was:

Split	**Images**
Training	2,750
Validation	589
Test	590
Total	**3,929**

No overlap was found between the three image sets.

The number of annotated instances in each split was:

Split	**Excavator**	**Dozer**	**Dump Truck**
Train	1,943	806	2,618
Validation	418	161	528
Test	429	156	555

The same image-level split was used across the detector families to maintain consistency in evaluation.

Two – Stage Object Detection

Faster R-CNN

Faster R-CNN was selected as the two-stage object detector for this experiment. Faster R-CNN introduces a **Region Proposal Network (RPN)** that shares convolutional features with the detection network. The RPN generates candidate object regions, after which the Fast R-CNN detection component performs classification and bounding-box refinement [1].

The architecture is suitable for this experiment because construction machinery can occur at different scales and locations within the images. The region proposal mechanism followed by region-level refinement provides explicit localization of candidate objects. This makes Faster R-CNN a suitable choice for evaluating object detection performance on the ACID dataset.

The original Faster R-CNN work demonstrated approximately **5 frames per second (FPS)** using a VGG-16-based implementation while achieving strong object detection accuracy, establishing the architecture as an important two-stage object detection framework [1].

Model Configuration

The model used was:

Faster R-CNN + ResNet-50-FPN

The model was initialized with **COCO-pretrained weights** and fine-tuned for the three ACID classes.

Training Configuration

Parameter	Configuration
Model	Faster R-CNN ResNet-50-FPN
Pretrained weights	COCO
Classes	3
Batch size	2
Epochs	10
Optimizer	SGD
Learning rate	0.005
Momentum	0.9
Weight decay	0.0005
LR scheduler	StepLR
Step size	3 epochs
LR decay	0.1
Hardware	NVIDIA GeForce RTX 4070 Laptop GPU
Training time	95.01 min

No additional image augmentation was introduced.

Training and Validation Loss

The Faster R-CNN training loss decrease d consistently during training.

Epoch	Training Loss	Validation Loss
1	0.2512	0.1938
2	0.1709	0.1730
3	0.1406	0.1634
4	0.0980	0.1536
5	0.0886	0.1516
6	0.0833	0.1552
7	0.0777	0.1564
8	0.0770	0.1566
9	0.0767	0.1570
10	0.0761	0.1571

The training loss continued to decrease throughout the ten epochs. The validation loss decreased rapidly during the first five epochs and then remained approximately stable. The small increase after epoch 5 indicates that further optimization primarily improved training loss without producing a corresponding improvement in validation loss.

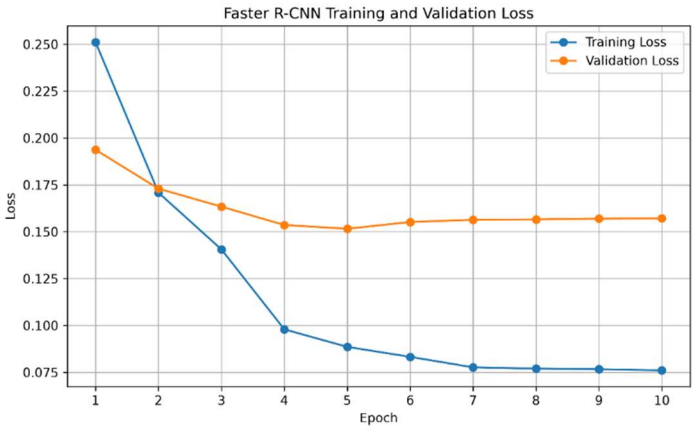


Fig 7. Faster R-CNN training and validation loss curve

Faster R-CNN Quantitative Results

The validation results were:

Metric	Result
mAP@0.5	**0.9203**
mAP@0.5:0.95	**0.6931**

The per-class AP values reported by the evaluation were:

Class	AP@0.5:0.95
Excavator	0.7184
Dozer	0.7382
Dump Truck	0.6227

The Dump Truck class has the lowest AP among the three classes despite having the largest number of training instances. This suggests that class frequency alone does not determine detection performance and that factors such as appearance variation, object scale, occlusion, and localization difficulty can also affect performance.

Faster R-CNN Qualitative Results

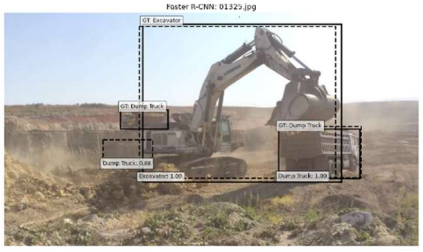
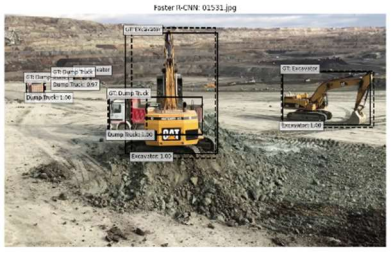
Six qualitative examples were generated:

- Three successful detections
- Three failure cases





Figs 8–10. Successful Faster R-CNN detections

Figs 11–13. Failed Faster R-CNN detections

The successful examples demonstrate that Faster R-CNN can localize construction equipment accurately when sufficient visual information is available. The failure examples demonstrate that detection can still degrade in the presence of difficult object configurations, scale variation, occlusion, or visually ambiguous regions.

One – Stage Object Detection

YOLOv8

YOLOv8s was selected as the one-stage object detector for this experiment. One-stage detectors perform object localization and classification in a single detection pipeline without a separate region-proposal stage, making them attractive for applications where high inference throughput is important [2] .

YOLOv8 was selected because the assignment requires comparison across different object detection families, while construction-site monitoring is a potential real-time application. The smaller **YOLOv8s** configuration provides a practical balance between computational requirements and detection accuracy, making it suitable for experimentation on the available hardware [2] .

It is important to note that Ultralytics does **not provide a formal research paper specifically for YOLOv8**. Therefore, the official Ultralytics documentation and software repository are used as the primary references for the YOLOv8 model and its implementation [2] .

Model Configuration

The model was initialized using **YOLOv8s COCO-pretrained weights** and fine-tuned on the three ACID classes.

Parameter	Configuration
Model	YOLOv8s
Initialization	COCO pretrained
Classes	3
Input size	640 × 640
Batch size	2
Epochs	10
Optimizer	SGD
Learning rate	0.005
Momentum	0.9
Weight decay	0.0005
Hardware	NVIDIA GeForce RTX 4070 Laptop GPU
Training time	23.97 min
Additional augmentation	Disabled
Random seed	42

The same image-level split was used as Faster R-CNN, and additional augmentation was disabled.

YOLOv8 Quantitative Results

The validation results were:

Class	Precision	Recall	AP@0.5	AP@0.5:0.95
Excavator	0.948	0.835	0.910	0.741
Dozer	0.937	0.836	0.930	0.743
Dump Truck	0.865	0.777	0.855	0.631
Overall	**0.917**	**0.816**	**0.898**	**0.705**

Thus:

- **mAP@0.5 = 0.8983**
- **mAP@0.5:0.95 = 0.7050**

YOLOv8 achieves a slightly lower mAP@0.5 than Faster R-CNN but a slightly higher mAP@0.5:0.95.

This indicates that YOLOv8 provides competitive localization quality at stricter IoU thresholds while requiring substantially less training time and fewer parameters.

YOLOv8 Inference Speed

The measured inference speed was approximately:

49.64 FPS

The validation run itself reported approximately 6.9 ms of model inference per image, while the dedicated 100-image timing experiment produced the approximately 49.64 FPS comparison value.

The model contains approximately:

11.14 million parameters.

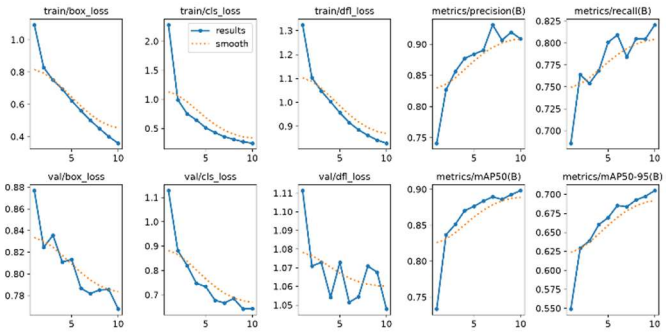


Fig 14. YOLOv8 training results

The generated precision-recall, precision, recall, F1, and confusion-matrix plots can also be included:

- BoxP_curve.png
- BoxR_curve.png
- BoxF1_curve.png
- BoxPR_curve.png
- confusion_matrix.png
- confusion_matrix_normalized.png

YOLOv8 Failure Analysis

Six qualitative examples were retained from the validation experiment:

- Three successful detections
- Three failed detections

The generated examples should be inserted **without replacing or hiding any failure cases**, as required by the assignment.



Fig 15–17. YOLOv8 successful examples



Fig 18–20. YOLOv8 failure examples

Potential causes of the observed failures include:

Observation

The validation results show that Dump Truck has the lowest AP among the three classes:

$$AP@0.5{:}0.95 = 0.6309$$

compared with:

- Excavator = 0.7411
- Dozer = 0.7429

The Dump Truck class therefore represents the most difficult category for YOLOv8 in this experiment.

Possible Explanation

The lower performance may be related to visual variation, object scale, partial occlusion, background similarity, or differences in viewpoint.

The input resolution was fixed at **640 × 640**, meaning objects that occupy relatively small regions of the original image may lose fine spatial information after resizing.

The class distribution is also imbalanced, with Dump Truck being the most frequent class but still exhibiting the lowest AP. Therefore, the failure cannot simply be attributed to insufficient Dump Truck training examples.

The experiment used YOLOv8's detection and post-processing pipeline without introducing custom anchor-size modifications or NMS tuning. These parameters could potentially affect crowded or overlapping construction-machine detections.

These points should be presented as **possible explanations rather than definitive causes**, since an ablation study was not performed to isolate each factor.

Transformer – Based Object Detection

DETR

The **Detection Transformer (DETR)** was selected as the transformer-based detector. DETR formulates object detection as a direct set-prediction problem. It uses a transformer encoder-decoder architecture to model relationships between image features and object queries, while bipartite matching is used to establish a one-to-one correspondence between predicted and ground-truth objects [3]. Unlike conventional anchor-based detectors, DETR does not require hand-designed anchor boxes or a separate non-maximum suppression (NMS) stage during inference [3].

DETR was selected because it provides a fundamentally different detection paradigm from Faster R-CNN and YOLOv8. While Faster R-CNN uses a region-proposal mechanism and YOLOv8 performs one-stage detection, DETR treats object detection as a direct set-prediction problem using transformer-based global attention. This makes it suitable for studying the application of transformer-based global reasoning to construction-equipment detection [3].

The pretrained model used in this experiment was **facebook/detr-resnet-50**, which uses a ResNet-50 backbone with the DETR transformer architecture. The pretrained checkpoint was used as the initialization for fine-tuning on the ACID dataset.

Model Configuration

Parameter	Configuration
Model	DETR ResNet-50
Pretrained checkpoint	facebook/detr-resnet-50
Classes	3
Batch size	2
Epochs	5
Optimizer	AdamW
Learning rate	1×10^{-5}
Weight decay	1×10^{-4}
Hardware	NVIDIA GeForce RTX 4070 Laptop GPU
Training time	46.44 min

The DETR experiment used the same training/validation/test image split and did not introduce additional data augmentation.

DETR Training Convergence

The DETR training and validation losses were:

Epoch	Training Loss	Validation Loss
1	1.1712	1.0562
2	0.8654	0.7986
3	0.7336	0.7491
4	0.6658	0.6590
5	0.6045	0.5979

Both training and validation losses decreased consistently during the five epochs.

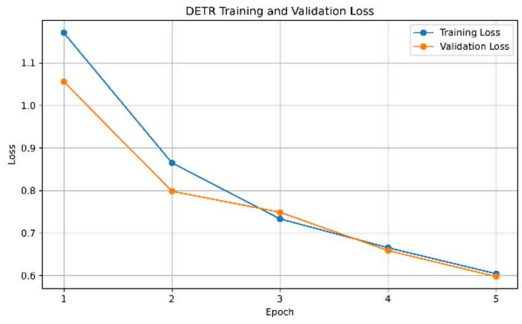


Fig 21. DETR training and validation loss curve

Although the loss converged during the five-epoch experiment, the resulting detection accuracy remained substantially lower than the CNN-based detectors.

This behaviour is consistent with a known characteristic of the original DETR architecture: vanilla DETR can require considerably longer training to reach strong detection performance compared with established CNN-based detectors. Subsequent research has specifically investigated the slow convergence and small-object limitations of DETR.

Therefore, the five-epoch DETR result should be interpreted as the performance of the **implemented training configuration**, rather than evidence that DETR cannot achieve higher performance with longer training.

DETR Quantitative Results

The validation results were:

Metric	Result
mAP@0.5	**0.5391**
mAP@0.5:0.95	**0.4152**

The reported per-class AP values were:

Class	AP@0.5:0.95
Excavator	0.5746
Dozer	0.1260
Dump Truck	0.5449

The most significant weakness is the **Dozer class**, where AP is only **0.1260**. In comparison, Excavator and Dump Truck achieve substantially higher values. This suggests that the DETR model had difficulty learning a reliable representation for the Dozer category under the short training schedule.

DETR Inference Speed

The dedicated inference benchmark evaluated 100 images.

- Images evaluated: 100
- Total inference time: 6.44 s
- Inference speed: **15.54 FPS**

The comparison benchmark used the same timing procedure across the three detectors and produced:

$$DETR = 18.02\ FPS$$

The latter value should be used in the final comparison table because it comes from the common comparison benchmark.

The DETR model contains approximately:

41.50 million parameters.

DETR Qualitative Results

Three successful and three failed examples were generated.



Fig 22–24. DETR successful examples



Fig 25–27. DETR failed examples

Attention maps were not generated as part of the implemented pipeline. Therefore, no attention-map-based conclusion should be claimed.

The failure results, particularly the very low Dozer AP, indicate that the short five-epoch training schedule was insufficient for DETR to learn all three construction-equipment categories equally well.

Experimental Comparison

The three detector families were compared using:

- mAP@0.5
- mAP@0.5:0.95
- Per-class AP
- Inference FPS

- Number of parameters
- Training time

Quantitative Comparison

Detector	mAP@0.5	mAP@0.5 :0.95	Excavator AP	Dozer AP	Dump Truck AP	FPS	Parameters	Training Time
Faster R-CNN ResNet-50-FPN	**0.9203**	0.6931	0.7184	0.7382	0.6227	15.73	41.36 M	95.01 min
YOLOv8s	0.8983	**0.7050**	**0.7411**	**0.7429**	**0.6309**	**49.64**	**11.14 M**	**23.97 min**
DETR ResNet-50	0.5391	0.4152	0.5746	0.1260	0.5449	18.02	41.50 M	46.44 min

Important note on the per-class values

The Faster R-CNN and DETR implementations recorded map_per_class from TorchMetrics for the overall map metric, corresponding to **AP averaged over IoU thresholds from 0.50 to 0.95**.

For YOLOv8, the validation output directly provides both per-class AP@0.5 and AP@0.5:0.95.

Therefore, the table above uses the consistently available **AP@0.5:0.95 per-class values** for the cross-model comparison.

Accuracy Comparison

Faster R-CNN achieved the highest **mAP@0.5**:

$$\mathbf{0.9203}$$

followed by YOLOv8s:

$$\mathbf{0.8983}$$

and DETR:

$$\mathbf{0.5391}.$$

However, at the stricter mAP@0.5:0.95 metric, YOLOv8s achieved the highest value:

$$\mathbf{0.7050}$$

compared with:

- Faster R-CNN: 0.6931
- DETR: 0.4152

This indicates that YOLOv8s provided the best overall balance of localization accuracy across the stricter IoU range.

Per-Class Comparison

YOLOv8s produced the strongest per-class AP among the three models for the reported comparison:

- Excavator: **0.7411**
- Dozer: **0.7429**
- Dump Truck: **0.6309**

Faster R-CNN achieved similar performance:

- Excavator: 0.7184
- Dozer: 0.7382
- Dump Truck: 0.6227

DETR performed considerably worse, particularly on Dozer:

- Excavator: 0.5746
- Dozer: **0.1260**
- Dump Truck: 0.5449

The strong difference for Dozer indicates that the transformer model was not able to learn the category as effectively under the given training configuration.

Inference Speed

YOLOv8s was clearly the fastest detector in the common benchmark:

49.64 FPS

while Faster R-CNN and DETR achieved:

- Faster R-CNN: 15.73 FPS
- DETR: 18.02 FPS

Therefore, YOLOv8s provides approximately three times the inference throughput of Faster R-CNN in this experiment.

This makes YOLOv8s the most suitable model for applications where images or video frames need to be processed continuously.

Parameter Count

The parameter counts were:

Detector	Parameters
Faster R-CNN ResNet-50-FPN	41.36 M
YOLOv8s	11.14 M
DETR ResNet-50	41.50 M

YOLOv8s therefore uses substantially fewer parameters than either Faster R-CNN or DETR.

This reduced parameter count contributes to its practical suitability for real-time deployment.

Training Time

The measured training times were:

Detector	Training Time
Faster R-CNN	95.01 min
YOLOv8s	23.97 min
DETR	46.44 min

Faster R-CNN required the longest training time.

YOLOv8s completed training in approximately one quarter of the Faster R-CNN training time.

DETR required 46.44 minutes for only five epochs, which demonstrates the relatively high computational cost per training epoch.

However, the training-time comparison should be interpreted together with the number of epochs because Faster R-CNN and YOLOv8s were trained for 10 epochs, whereas DETR was trained for 5 epochs.

Speed–Accuracy Comparison

The speed–accuracy scatter plot uses:

- X-axis: Inference FPS
- Y-axis: mAP@0.5:0.95

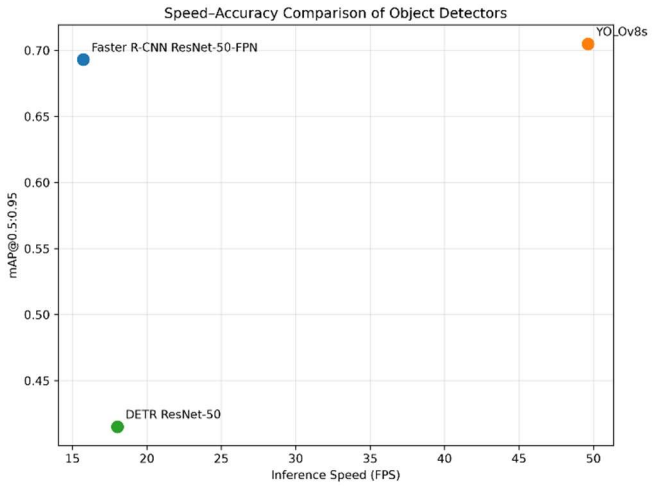


Fig 28. Speed–accuracy comparison of object detectors

The results show a clear trade-off between accuracy and inference speed.

YOLOv8s occupies the most favourable region of the plot because it provides both the highest mAP@0.5:0.95 and the highest inference speed among the three models.

Faster R-CNN achieves the highest mAP@0.5 but has significantly lower inference speed.

DETR does not provide a competitive speed–accuracy trade-off in the current experiment because its accuracy is substantially lower than both CNN-based detectors.

Deployment Discussion

Real-Time Construction-Site Monitoring

For real-time construction-site monitoring, **YOLOv8s is the preferred model**.

The main reasons are:

- Highest measured inference speed: **49.64 FPS**
- Highest mAP@0.5:0.95: **0.7050**
- Lowest parameter count: **11.14 M**
- Lowest training time: **23.97 min**
- Strong performance across all three classes

The combination of approximately 50 FPS and competitive accuracy makes YOLOv8s the most practical detector for continuous video-based monitoring.

This is particularly relevant to construction applications where machine detection can support productivity monitoring, equipment tracking, and safety-related analysis. The original ACID work also identifies construction-machine detection as a basis for automated construction monitoring.

Offline Batch Analysis

For offline analysis, **Faster R-CNN** can be considered when high mAP@0.5 is the primary objective.

Faster R-CNN achieved:

$$\boldsymbol{mAP@0.5 = 0.9203}$$

which was the highest among the three models.

However, this comes at the cost of:

- 41.36 M parameters
- 15.73 FPS
- 95.01 min training time

Therefore, Faster R-CNN is better suited to situations where processing speed is less important and detection quality at IoU 0.5 is prioritized.

YOLOv8s remains preferable when both accuracy and processing throughput are important.

Limitations

Several limitations were observed during the experiment.

1. Limited Number of Classes

Although the original ACID dataset contains ten construction-machine categories, this assignment used only three:

- Excavator
- Dozer
- Dump Truck

Therefore, the results cannot be generalized to the complete ACID dataset.

2. Class Imbalance

The number of instances varies substantially between the three classes.

Dump Truck has 3,701 instances, while Dozer has only 1,123.

This imbalance can influence the learned detector and may contribute to differences in per-class performance.

3. No Additional Data Augmentation

Additional augmentation was intentionally disabled to maintain consistency between the experiments.

Although this improves experimental control, augmentation could potentially improve robustness to viewpoint, illumination, scale, and occlusion variations.

4. DETR Training Duration

DETR was trained for only five epochs.

Although both training and validation losses decreased throughout the experiment, the detection performance remained substantially below the CNN-based detectors.

The original DETR architecture is known to have slower convergence characteristics, and subsequent research has proposed modifications specifically to address slow convergence.

Therefore, longer DETR training could potentially improve its performance.

5. No Attention Map Analysis

The assignment suggested visualizing attention maps for transformer failure cases where possible.

Attention maps were not generated in the implemented DETR pipeline. Therefore, the transformer failure analysis is based on detection outputs and quantitative results rather than direct attention visualization.

6. Different Training Configurations

Faster R-CNN and YOLOv8 were trained for 10 epochs, whereas DETR was trained for 5 epochs.

Therefore, training-time comparisons should not be interpreted as a perfectly normalized measure of optimization efficiency.

The inference-speed comparison, however, was performed using a common benchmark procedure on the validation images.

7. Validation-Based Comparison

The quantitative detector comparison is based on the validation split.

The test set was retained but was not used as the primary model-comparison set in the reported experiment.

8. Failure Analysis Without Controlled Ablations

Possible explanations such as input resolution, object scale, class imbalance, and NMS behaviour were considered when analyzing failures.

However, these factors were not individually isolated through controlled ablation experiments. Therefore, they should be regarded as possible explanations rather than experimentally proven causes.

Conclusion

This assignment implemented and compared three object detection approaches on the three-class ACID construction-equipment dataset: Faster R-CNN, YOLOv8s, and DETR.

The dataset contained **3,929 images and 7,614 annotated construction-equipment instances**, divided into training, validation, and test sets using a 70:15:15 image-level split.

Faster R-CNN achieved the highest **mAP@0.5 of 0.9203**, demonstrating strong detection performance. Its two-stage region proposal and refinement architecture provides accurate object localization but requires a relatively large computational budget and resulted in the longest training time.

YOLOv8s achieved the highest **mAP@0.5:0.95 of 0.7050** and the highest inference speed of **49.64 FPS**. It also had the smallest parameter count at approximately **11.14 million parameters** and the shortest training time of **23.97 minutes**. These results make YOLOv8s the strongest overall choice for real-time construction-site monitoring.

DETR provided a different transformer-based detection paradigm using global attention and set-based prediction. Its validation loss decreased consistently during training, but the five-epoch experiment resulted in substantially lower detection performance, with **mAP@0.5 = 0.5391** and **mAP@0.5:0.95 = 0.4152**. The particularly low Dozer AP indicates difficulty learning this category under the current training configuration.

Overall, the experiment demonstrates that there is no single detector that is optimal for every application. **Faster R-CNN is preferable when detection accuracy at IoU 0.5 is prioritized, while YOLOv8s provides the best accuracy–speed trade-off for real-time deployment. DETR demonstrates the potential of transformer-based detection but requires further training and potentially architectural improvements to become competitive on this dataset.**

References

[1] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in *Advances in Neural Information Processing Systems*, vol. 28, 2015.

[2] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," version 8.0.0, Ultralytics, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[3] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers," in *Computer Vision – ECCV 2020*, Cham, Switzerland: Springer, 2020, pp. 213–229, doi: 10.1007/978-3-030-58452-8_13.

[4] B. Xiao and S. C. Kang, "Development of an image data set of construction machines for deep learning object detection," *Journal of Computing in Civil Engineering*, vol. 35, no. 2, Art. no. 05020005, 2021, doi: 10.1061/(ASCE)CP.1943-5487.0000945.

[5] X. Zhu, W. Su, L. Lu, B. Li, X. Wang, and J. Dai, "Deformable DETR: Deformable transformers for end-to-end object detection," in *International Conference on Learning Representations (ICLR)*, 2021.

[6] Hugging Face, "facebook/detr-resnet-50," *Hugging Face Model Hub*. [Online]. Available: https://huggingface.co/facebook/detr-resnet-50

Coding References

[7] PyTorch, "PyTorch: An open source machine learning framework," PyTorch Foundation. [Online]. Available: https://pytorch.org/

[8] PyTorch, "Torchvision object detection finetuning tutorial," PyTorch Documentation. [Online]. Available: https://docs.pytorch.org/tutorials/intermediate/torchvision_tutorial.html

[9] Ultralytics, "Ultralytics YOLO documentation," Ultralytics. [Online]. Available: https://docs.ultralytics.com/

[10] Ultralytics, "YOLOv8 documentation," Ultralytics. [Online]. Available: https://docs.ultralytics.com/models/yolov8/

[11] Hugging Face, "Transformers documentation," Hugging Face. [Online]. Available: https://huggingface.co/docs/transformers/

[12] Hugging Face, "DETR documentation," Hugging Face. [Online]. Available: https://huggingface.co/docs/transformers/model_doc/detr

[13] Hugging Face, "facebook/detr-resnet-50," Hugging Face Model Hub. [Online]. Available: https://huggingface.co/facebook/detr-resnet-50

[14] OpenCV, “OpenCV documentation,” OpenCV. [Online]. Available: https://docs.opencv.org/

[15] Matplotlib Development Team, “Matplotlib documentation,” Matplotlib. [Online]. Available: https://matplotlib.org/stable/

[16] NumPy Developers, “NumPy documentation,” NumPy. [Online]. Available: https://numpy.org/doc/

GitHub Link: https://github.com/vasantharan/CV5570_Assignment_2**

**Repo will be made public after submission deadline

- END -